TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

WEBSITE HỌC TIẾNG ANH
[TÊN DỰ ÁN]

(Đồ án môn học Lập trình trên web)

Nhóm thực hiện: [Số nhóm]

STT	Họ tên	MSSV
1	[Họ tên]	[MSSV]
2	[Họ tên]	[MSSV]
3	[Họ tên]	[MSSV]
4	[Họ tên]	[MSSV]

Giảng viên hướng dẫn

STT	Họ tên, học hàm, học vị
1	[Giảng viên 1]
2	[Giảng viên 2]

TP. Hồ Chí Minh, ngày [..] tháng [..] năm 2026

MỤC LỤC

1. Giới thiệu

    1.1. Tên đề tài

    1.2. Mục đích

    1.3. Chức năng chính

    1.4. Đối tượng sử dụng

2. Phân tích và thiết kế

    2.1. Kiến trúc hệ thống

    2.2. Công nghệ sử dụng

    2.3. Cơ sở dữ liệu

    2.4. Giao diện người dùng

    2.5. API và luồng dữ liệu

3. Triển khai

    3.1. Cấu trúc thư mục dự án

    3.2. Các module chính

    3.3. Xử lý nghiệp vụ

    3.4. Bảo mật

4. Kết quả và đánh giá

    4.1. Kết quả đạt được

    4.2. Hạn chế

    4.3. Hướng phát triển

Tài liệu tham khảo

Phân công công việc

1. Giới thiệu

1.1. Tên đề tài

Xây dựng website học tiếng Anh hỗ trợ đọc hiểu, từ vựng và theo dõi tiến độ học tập.

1.2. Mục đích

Website giúp người học tự học tiếng Anh qua các hoạt động: đọc bài theo chủ đề, học từ vựng có phát âm, làm bài kiểm tra từ vựng và theo dõi chuỗi ngày học liên tục. Mục tiêu là tạo môi trường học tập tự chủ, giao diện đơn giản, dễ tiếp cận trên mọi thiết bị.

1.3. Chức năng chính

- **Đăng ký và đăng nhập:** xác thực email bằng mã OTP 6 chữ số, captcha Cloudflare Turnstile chống bot.
- **Đọc hiểu:** danh sách bài đọc theo chủ đề, mỗi lần mở bài hệ thống ghi lượt xem.
- **Từ vựng:** học từ theo chủ đề, xem nghĩa tiếng Việt, nghe phát âm từ Cambridge Dictionary.
- **Quiz từ vựng:** hệ thống đưa nghĩa tiếng Việt, người dùng viết lại từ tiếng Anh, máy chủ chấm điểm và lưu kết quả.
- **Theo dõi chuỗi ngày học:** hiển thị số ngày học liên tục, chuỗi dài nhất, lịch học từng tháng.
- **Quản trị nội dung:** quản trị viên thêm, sửa, xoá chủ đề, bài đọc, từ vựng.

1.4. Đối tượng sử dụng

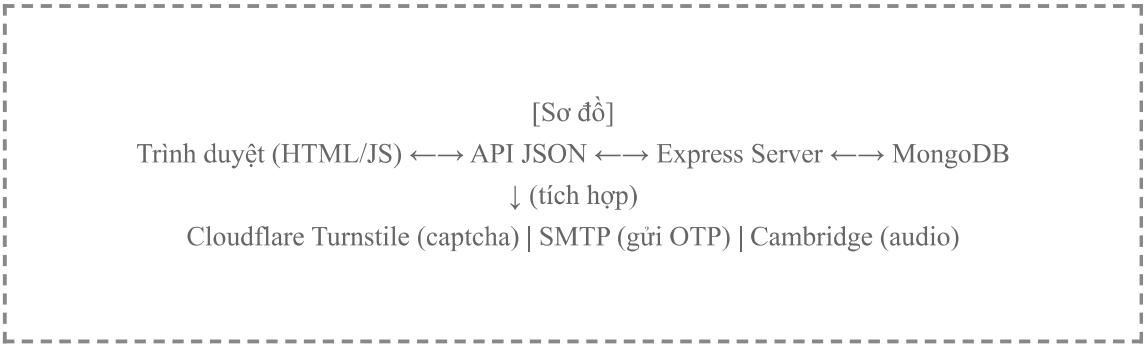
- **Học viên:** đăng ký tài khoản, sử dụng các chức năng học tập.
- **Quản trị viên:** quản lý nội dung học (thêm, sửa, xoá), được phân quyền theo email cấu hình.

2. Phân tích và thiết kế

2.1. Kiến trúc hệ thống

Hệ thống theo mô hình **Client-Server** với kiến trúc **RESTful API**:

- **Client (Front-end):** HTML, CSS, JavaScript thuần (không framework), chạy trong trình duyệt, gọi API để lấy và gửi dữ liệu dạng JSON.
- **Server (Back-end):** Node.js với Express framework, xử lý logic nghiệp vụ, xác thực, phân quyền, truy vấn cơ sở dữ liệu.
- **Database:** MongoDB (NoSQL), lưu trữ người dùng, nội dung học, kết quả quiz.



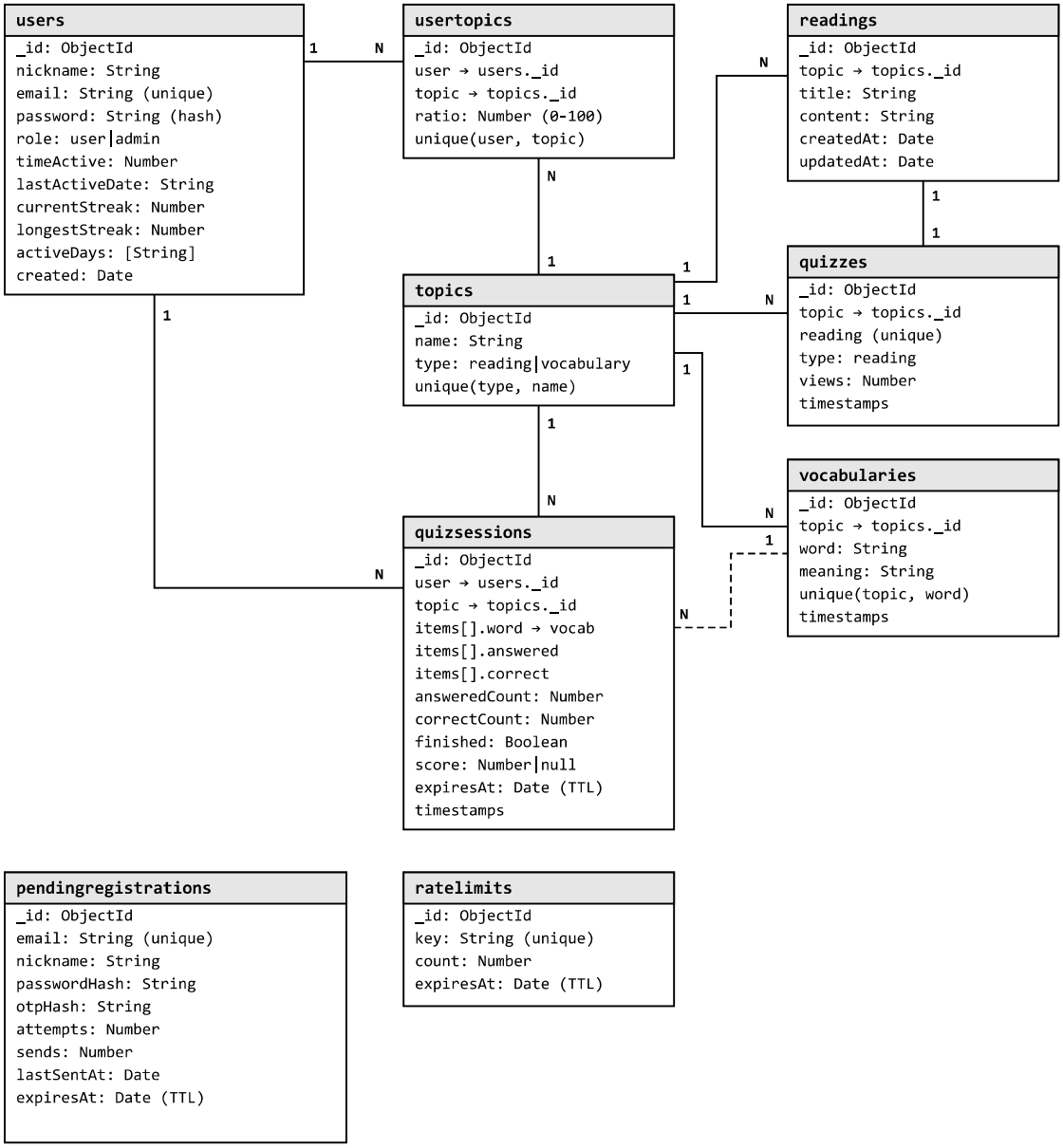
Hình 2.1. Kiến trúc tổng quan

2.2. Công nghệ sử dụng

Thành phần	Công nghệ
Ngôn ngữ lập trình	JavaScript (Node.js 18+ cho server, ES6+ cho client)
Web framework	Express 4.19
Cơ sở dữ liệu	MongoDB 6+ với Mongoose ODM 8
Xác thực	JSON Web Token (JWT), bcryptjs (băm mật khẩu)
Gửi email	nodemailer (SMTP)
Captcha	Cloudflare Turnstile
Audio phát âm	Cambridge Dictionary API (public)
Công cụ phát triển	[VS Code, Git, Postman...]
Triển khai	[Vercel/Railway/...], MongoDB Atlas

2.3. Cơ sở dữ liệu

Hệ thống sử dụng MongoDB với 9 collection. Quan hệ giữa các collection được thể hiện qua trường tham chiếu ObjectId.



Hình 2.2. Sơ đồ cơ sở dữ liệu (TTL: tự động xoá khi hết hạn)

Collection	Mô tả
users	Lưu tài khoản, vai trò, thông tin streak (chuỗi ngày học)
topics	Chủ đề dùng chung cho đọc hiểu và từ vựng

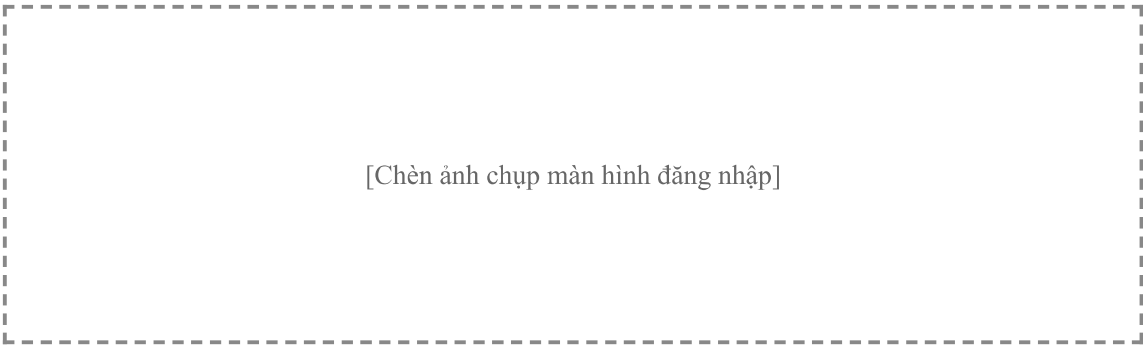
readings	Nội dung bài đọc
vocabularies	Từ tiếng Anh, nghĩa tiếng Việt theo chủ đề
quizzes	Thống kê lượt mở bài đọc (mỗi bài 1 bản ghi)
quizsessions	Phiên làm quiz từ vựng, tự xoá sau 120 phút
usertopics	Điểm quiz gần nhất của mỗi người theo chủ đề
pendingregistrations	Đăng ký đang chờ nhập OTP, tự xoá khi hết hạn
ratelimits	Giới hạn số lần gọi API theo IP

2.4. Giao diện người dùng

Website có 2 trang HTML chính:

- `login.html` – Đăng nhập, đăng ký, nhập mã OTP
- `index.html` – Trang học với 4 tab: Đọc hiểu, Từ vựng, Làm quiz, Chuỗi ngày

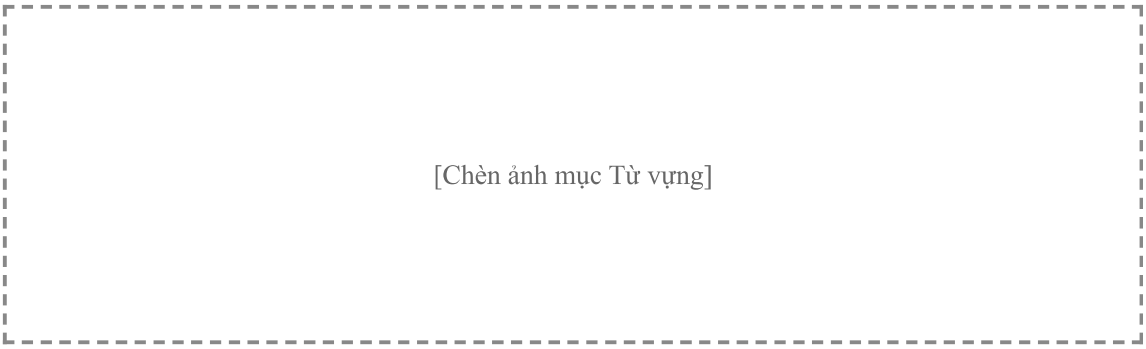
Giao diện responsive, sử dụng CSS Grid và Flexbox, hoạt động tốt trên mobile. Các màn hình tiêu biểu:



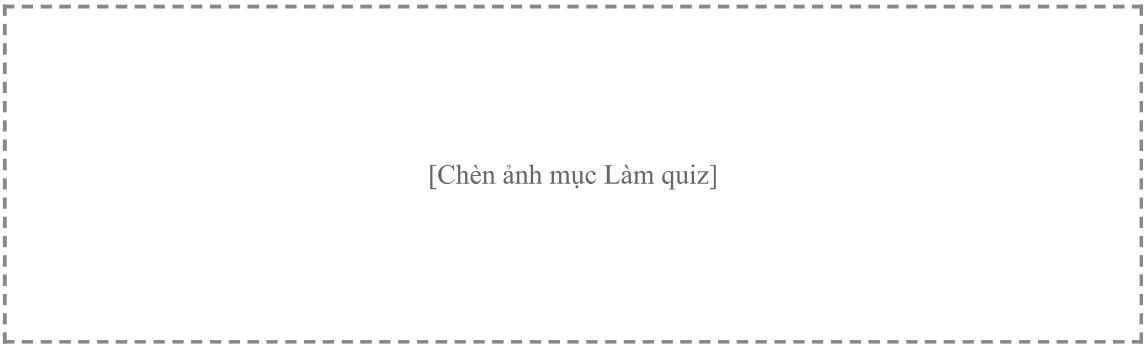
Hình 2.3. Trang đăng nhập



Hình 2.4. Mục Đọc hiểu



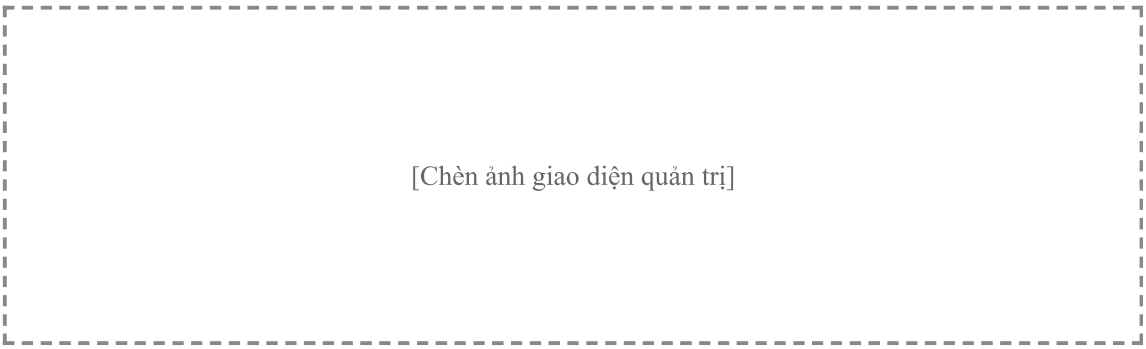
Hình 2.5. Mục Từ vựng



Hình 2.6. Làm quiz từ vựng



Hình 2.7. Theo dõi chuỗi ngày học



Hình 2.8. Giao diện quản trị (thêm/sửa/xoá nội dung)

2.5. API và luồng dữ liệu

Server cung cấp RESTful API, tất cả response đều là JSON. Front-end gọi API bằng `fetch()`.

Các endpoint chính

Endpoint	Chức năng
POST /api/auth/register	Đăng ký (captcha → gửi OTP qua email)
POST /api/auth/verify-otp	Xác thực OTP, tạo tài khoản
POST /api/auth/login	Đăng nhập, trả JWT token
GET /api/auth/me	Lấy thông tin user hiện tại
GET /api/reading/topics	Danh sách chủ đề đọc hiểu
GET /api/reading	Danh sách bài đọc (có thể lọc theo topic)
GET /api/reading/:id	Chi tiết bài đọc, ghi lượt xem, cập nhật streak
POST /api/reading (admin)	Thêm bài đọc mới
PUT /api/reading/:id (admin)	Sửa bài đọc
DELETE /api/reading/:id (admin)	Xoá bài đọc
GET /api/vocabulary/topics	Danh sách chủ đề từ vựng
GET /api/vocabulary/topics/:id	Danh sách từ theo chủ đề (có audio URL)
GET /api/vocabulary/topics/:id/words	Bắt đầu quiz: tạo session, trả nghĩa tiếng Việt
POST /api/vocabulary/quiz/:sessionId/check	Chấm 1 câu trả lời
POST /api/vocabulary/quiz/:sessionId/finish	Nộp bài, tính điểm, lưu kết quả, cập nhật streak
GET /api/streak	Thông tin chuỗi ngày học, lịch học

Luồng dữ liệu ví dụ: Làm quiz từ vựng

```
1. User chọn chủ đề → GET /api/vocabulary/topics/:id/words
   ← Server: tạo QuizSession, xáo trộn từ, trả { sessionId, words: {id: nghĩa} }

2. Với mỗi từ, user nhập đáp án → POST /api/vocabulary/quiz/:sessionId/check
   Body: { wordId, answer }
   ← Server: chuẩn hoá đáp án, so sánh, cập nhật session, trả { correct: true/false }

3. User nộp bài hoặc hết từ → POST /api/vocabulary/quiz/:sessionId/finish
   ← Server: tính điểm, lưu vào UserTopic, cập nhật streak, trả kết quả đầy đủ
```

3. Triển khai

3.1. Cấu trúc thư mục dự án

```
webtienganh-backend/
├── public/              # Front-end (HTML, CSS, JS)
│   ├── index.html
│   ├── login.html
│   ├── styles.css
│   ├── core.js          # Auth, API client
│   ├── app.js           # Điều phối tab
│   ├── reading.js       # Tab đọc hiểu
│   ├── vocab.js         # Tab từ vựng
│   ├── quiz.js          # Tab làm quiz
│   └── streak.js        # Tab chuỗi ngày
├── src/
│   ├── models/          # Mongoose schemas
│   │   ├── User.js
│   │   ├── Topic.js
│   │   ├── Reading.js
│   │   ├── Vocabulary.js
│   │   ├── Quiz.js
│   │   ├── QuizSession.js
│   │   ├── UserTopic.js
│   │   ├── PendingRegistration.js
│   │   └── RateLimit.js
│   ├── controllers/     # Logic xử lý nghiệp vụ
│   │   ├── auth.js
│   │   ├── reading.js
│   │   ├── vocabulary.js
│   │   ├── vocabularyQuiz.js
│   │   └── streak.js
│   ├── routes/          # Định nghĩa endpoint
│   │   ├── index.js
│   │   ├── auth.js
│   │   ├── reading.js
│   │   ├── vocabulary.js
│   │   └── streak.js
│   ├── middlewares/     # Middleware
│   │   ├── auth.js      # authRequired, adminRequired
│   │   └── errorHandler.js
│   ├── utils/           # Các hàm tiện ích
│   │   ├── asyncHandler.js
│   │   ├── httpError.js
│   │   ├── mailer.js
│   │   ├── turnstile.js
│   │   ├── audio.js
│   │   ├── date.js
│   │   ├── streak.js
│   │   └── validate.js
│   ├── config/          # Cấu hình
```

```
│   │   └── index.js
│   ├── scripts/        # Script hỗ trợ
│   │   └── seed.js     # Nhập dữ liệu mẫu
│   ├── app.js          # Khởi tạo Express app
│   └── server.js       # Entry point
├── .env                # Biến môi trường
├── package.json
└── vercel.json         # Cấu hình deploy Vercel
```

3.2. Các module chính

Models (Mongoose Schema)

Định nghĩa cấu trúc dữ liệu, ràng buộc, quan hệ giữa các collection. Ví dụ model `Reading`:

```
const readingSchema = new mongoose.Schema({
  topic: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Topic',
    required: true,
    index: true
  },
  title: { type: String, required: true, trim: true, maxlength: 200 },
  content: { type: String, required: true, maxlength: 20000 }
}, { timestamps: true });

module.exports = mongoose.model('Reading', readingSchema);
```

Controllers

Xử lý logic nghiệp vụ, giao tiếp với models, trả response. Ví dụ controller bắt đầu quiz từ vựng (`vocabularyQuiz.js`):

```
exports.startQuiz = asyncHandler(async (req, res) => {
  const topic = await findTopic(req.params.topicId);
  const words = await Vocabulary.find({ topic: topic._id })
    .select('meaning').lean();
  if (!words.length) throw new HttpError(404, 'Chủ đề chưa có từ vựng');

  const shuffled = shuffle(words);  // Xáo trộn thứ tự
  const session = await QuizSession.create({
    user: req.userId || null,
    topic: topic._id,
    items: shuffled.map(w => ({
      word: w._id,
      answered: false,
      correct: false
    })),
    answeredCount: 0,
    correctCount: 0,
    finished: false,
    expiresAt: new Date(Date.now() + config.quizSessionTtlMinutes * 60000)
  });

  // Chỉ trả nghĩa tiếng Việt, không trả từ tiếng Anh (tránh lộ đáp án)
  const wordsMap = {};
  shuffled.forEach(w => { wordsMap[w._id] = w.meaning; });
  res.json({ sessionId: session._id, words: wordsMap });
});
```

Routes

Định nghĩa endpoint, gắn middleware xác thực, gọi controller. Ví dụ `routes/reading.js`:

```
const router = require('express').Router();
const { authOptional, adminRequired } = require('../middlewares/auth');
const ctrl = require('../controllers/reading');

router.get('/topics', ctrl.getTopics);
router.get('/', ctrl.getReadings);
router.get('/:id', authOptional, ctrl.getReading);  // Ghi streak nếu đăng
nhập
router.post('/', adminRequired, ctrl.createReading);
router.put('/:id', adminRequired, ctrl.updateReading);
router.delete('/:id', adminRequired, ctrl.deleteReading);

module.exports = router;
```

Middlewares

authRequired: đọc JWT từ header `Authorization: Bearer <token>`, verify, gán `req.userId`. Nếu token thiếu hoặc không hợp lệ → trả 401.

adminRequired: gọi `authRequired`, sau đó kiểm tra `User.role === 'admin'`. Không phải admin → trả 403.

errorHandler: bắt mọi lỗi, trả JSON `{ error: message }` với status code phù hợp.

Front-end (Single Page Application thủ công)

Không dùng framework. Mỗi tab có một file JS riêng, gọi hàm `api()` trong `core.js` để gọi server, sau đó dựng HTML bằng template string và gán vào DOM:

```
// core.js
async function api(method, path, body) {
  const opt = { method, headers: {} };
  const token = localStorage.getItem('token');
  if (token) opt.headers.Authorization = 'Bearer ' + token;
  if (body) {
    opt.headers['Content-Type'] = 'application/json';
    opt.body = JSON.stringify(body);
  }
  const res = await fetch(apiBase + path, opt);
  if (!res.ok) {
    if (res.status === 401) {
      localStorage.removeItem('token');
      location.href = 'login.html';
    }
    const err = await res.json().catch(() => ());
    throw new Error(err.error || 'Lỗi kết nối');
  }
  return res.json();
}
```

3.3. Xử lý nghiệp vụ

Đăng ký với OTP

1. User điền form → client gọi Turnstile.getResponse() → POST /api/auth/register với captcha token
2. Server: xác thực captcha với Cloudflare, sinh mã OTP 6 chữ số, băm bằng bcrypt, lưu vào `pendingregistrations` (TTL 10 phút), gửi email
3. User nhập OTP → POST /api/auth/verify-otp
4. Server: so sánh hash, đúng thì tạo `User`, xoá pending, trả JWT

Chấm quiz từ vựng

Mỗi từ chỉ được trả lời 1 lần. Server chuẩn hoá đáp án (trim, lowercase, xoá khoảng trắng thừa), so sánh với từ gốc. Dùng `findOneAndUpdate` với điều kiện `items.$.answered === false` để đảm bảo không chấm lại:

```
const result = await QuizSession.findOneAndUpdate(
  { _id: sessionId, 'items._id': itemId, 'items.answered': false },
  {
    $set: {
      'items.$.answered': true,
      'items.$.correct': isCorrect
    },
    $inc: {
      answeredCount: 1,
      correctCount: isCorrect ? 1 : 0
    }
  },
  { new: true }
).lean();

if (!result) throw new HttpError(409, 'Câu này đã trả lời rồi');
```

Tính streak (chuỗi ngày học)

Hàm `touchStreak(userId)` chạy theo múi giờ UTC+7 (Việt Nam). Lấy ngày hiện tại dạng YYYY-MM-DD, so sánh với `lastActiveDate`:

- Cùng ngày → không làm gì
- Ngày liền kề → tăng `currentStreak`, cập nhật `longestStreak` nếu cần, thêm ngày vào `activeDays[]`
- Ngày gián đoạn → reset `currentStreak = 1`

Gọi khi: (1) đọc bài, (2) hoàn thành quiz.

Chống gian lận: tự động nộp bài khi chuyển tab

```
// quiz.js
document.addEventListener('visibilitychange', () => {
  if (document.hidden && quizState.active && !quizState.finished) {
    finishQuiz();  // Tự động nộp bài
  }
});
```

3.4. Bảo mật

- **Mật khẩu:** băm bằng bcryptjs (10 rounds) trước khi lưu, không bao giờ lưu plaintext.
- **JWT:** ký bằng secret key, thời gian sống 30 ngày, lưu trong localStorage (không dùng cookie).
- **OTP:** băm trước khi lưu vào database, giới hạn 5 lần thử sai, 60s giữa các lần gửi lại.
- **Rate limit:** giới hạn số lần gọi đăng ký, đăng nhập, OTP, quiz theo IP (dùng collection `ratelimits` với TTL).
- **Captcha:** Cloudflare Turnstile ở form đăng ký, server xác thực token với Cloudflare API.
- **CORS:** chỉ cho phép origin được cấu hình trong `.env` (production).
- **Phân quyền:** mọi API thêm/sửa/xoá đều qua middleware `adminRequired`.
- **XSS:** front-end escape mọi dữ liệu từ user trước khi chèn vào HTML (hàm `esc()`).
- **NoSQL Injection:** Mongoose tự động sanitize query, không dùng `$where`.

4. Kết quả và đánh giá

4.1. Kết quả đạt được

Dự án đã hoàn thành các chức năng chính theo yêu cầu đề tài:

- **Đăng ký và đăng nhập bảo mật:** xác thực email bằng OTP, captcha Turnstile chống bot, JWT lưu trạng thái đăng nhập.
- **Học từ vựng:** danh sách từ theo chủ đề, nghe phát âm từ Cambridge Dictionary, giao diện trực quan.
- **Quiz từ vựng:** viết lại từ từ nghĩa tiếng Việt, máy chủ chấm điểm theo thời gian thực, lưu kết quả theo chủ đề, chống gian lận (tự động nộp khi chuyển tab).
- **Đọc bài theo chủ đề:** hệ thống ghi lượt xem, cập nhật chuỗi ngày học khi mở bài.
- **Theo dõi tiến độ:** chuỗi ngày học liên tục, chuỗi dài nhất, lịch học từng tháng hiển thị trực quan.
- **Quản trị nội dung:** admin thêm/sửa/xoá chủ đề, bài đọc, từ vựng qua giao diện trên cùng một trang.
- **Responsive:** giao diện hoạt động tốt trên desktop và mobile.
- **Triển khai thành công:** deploy lên [Vercel/Railway/...], kết nối MongoDB Atlas, hoạt động ổn định.

4.2. Hạn chế

- **Chưa có phần Ngữ pháp:** thiết kế ban đầu có dự kiến nhưng chưa triển khai.
- **Chưa có trang quản trị riêng:** admin và user dùng chung giao diện `index.html`, giao diện admin chưa được tối ưu.
- **Chưa có tìm kiếm:** khi số lượng chủ đề, bài đọc nhiều, người dùng phải cuộn để tìm.
- **Chưa có phân trang:** danh sách trả về toàn bộ, có thể chậm khi dữ liệu lớn.
- **Âm thanh phát âm phụ thuộc vào Cambridge:** nếu Cambridge đổi URL hoặc chặn, âm thanh sẽ không phát được.
- **Chưa lưu lịch sử làm quiz:** chỉ lưu điểm gần nhất, không xem lại các lần trước.
- **Email OTP có thể vào spam:** tuỳ cấu hình SMTP và domain.

4.3. Hướng phát triển

- **Thêm kỹ năng Viết:** người dùng viết đoạn văn từ đề bài, hệ thống dùng AI (OpenAI API hoặc Claude API) chấm ngữ pháp, từ vựng, gợi ý sửa lỗi.
- **Thêm phần Ngữ pháp:** bài giảng ngữ pháp, quiz trắc nghiệm ngữ pháp theo chủ đề.

- **Trang quản trị riêng:** giao diện `admin.html` với bảng CRUD đầy đủ, thống kê số người dùng, bài đọc, top chủ đề được làm nhiều nhất.
- **Dữ liệu sinh bằng AI:** import nội dung từ tài liệu, dùng AI tóm tắt hoặc tạo câu hỏi; AI sinh bài đọc và từ vựng theo chủ đề người dùng nhập.
- **Tìm kiếm và phân trang:** tìm bài đọc, từ vựng theo từ khoá, phân trang danh sách.
- **Lưu lịch sử quiz:** xem lại điểm từng lần làm, biểu đồ tiến bộ theo thời gian.
- **Chế độ ôn tập:** hiện lại các từ đã trả lời sai trong quiz trước.
- **Âm thanh ghi âm:** người dùng đọc từ, hệ thống so sánh với phát âm chuẩn (Web Speech API).
- **Thông báo đẩy:** nhắc nhở học hàng ngày để giữ streak.
- **Chế độ tối (Dark mode):** cho người dùng học ban đêm.

TÀI LIỆU THAM KHẢO

1. Express.js Documentation. *https://expressjs.com/*
2. Mongoose Documentation. *https://mongoosejs.com/docs/*
3. MongoDB Manual. *https://www.mongodb.com/docs/manual/*
4. MDN Web Docs – JavaScript. *https://developer.mozilla.org/en-US/docs/Web/JavaScript*
5. Node.js Documentation. *https://nodejs.org/docs/*
6. JSON Web Tokens (JWT). *https://jwt.io/*
7. Cloudflare Turnstile Documentation. *https://developers.cloudflare.com/turnstile/*
8. Cambridge Dictionary. *https://dictionary.cambridge.org/*
9. bcrypt.js Documentation. *https://github.com/dcodeIO/bcrypt.js*
10. nodemailer Documentation. *https://nodemailer.com/*
11. [Nhóm bổ sung nguồn khác nếu có: sách, bài báo, StackOverflow, tutorial...]

PHÂN CÔNG CÔNG VIỆC

Thành viên	Nhiệm vụ chính	Tỷ lệ
[Họ tên 1] [MSSV]	[Backend: models (User, Topic, Reading, PendingRegistration, RateLimit), controllers (auth, reading), middlewares (authRequired, adminRequired), routes (/auth, /reading), utils (mailer, turnstile, validate), script seed, triển khai Vercel, cấu hình MongoDB Atlas]	[25%]
[Họ tên 2] [MSSV]	[Backend: models (Vocabulary, QuizSession, UserTopic, Quiz), controllers (vocabulary, vocabularyQuiz, streak), routes (/vocabulary, /streak), utils (audio, date, streak), logic chấm điểm quiz, xử lý streak múi giờ VN, API rate limit]	[25%]
[Họ tên 3] [MSSV]	[Frontend: login.html, login.js (đăng nhập, đăng ký, OTP, Turnstile), core.js (auth, api client, error handling), styles.css (layout chính, responsive mobile, tab navigation), kiểm thử giao diện trên nhiều thiết bị]	[25%]
[Họ tên 4] [MSSV]	[Frontend: reading.js (tab đọc hiểu), vocab.js (tab từ vựng, tích hợp audio Cambridge), quiz.js (tab làm quiz, logic chống chuyển tab), streak.js (tab chuỗi ngày, lịch học), app.js (điều phối tab), viết báo cáo đồ án]	[25%]

> **Lưu ý:** Nhóm điền tên, MSSV, mô tả nhiệm vụ và tỷ lệ thực tế của mỗi người. Tổng tỷ lệ phải bằng 100%.

Giảng viên hướng dẫn

[Họ tên, chữ ký]

TP. Hồ Chí Minh, ngày [..] tháng [..] năm 2026

Nhóm trưởng

[Họ tên, chữ ký]